

Assignment 5: Data Visualization

Alonso Velasquez

Fall 2025

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on Data Visualization

Directions

1. Rename this file `<FirstLast>_A05_DataVisualization.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure your code is tidy; use line breaks to ensure your code fits in the knitted output.
5. Be sure to **answer the questions** in this assignment document.
6. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up your session

1. Set up your session. Load the tidyverse, here & cowplot packages, and verify your home directory. Read in the NTL-LTER processed data files for nutrients and chemistry/physics for Peter and Paul Lakes (use the tidy NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Processed.csv version in the Processed_KEY folder) and the processed data file for the Niwot Ridge litter dataset (use the NEON_NIWO_Litter_mass_trap_Processed.csv version, again from the Processed_KEY folder).
2. Make sure R is reading dates as date format; if not change the format to date.

#1 Loading Packages

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    4.0.0      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(here)
```

```
## here() starts at /home/guest/R/EDE_Fall2025
```

```
library(cowplot)
```

```
##  
## Attaching package: 'cowplot'  
##  
## The following object is masked from 'package:lubridate':  
##  
## stamp
```

```
library(ggthemes)
```

```
##  
## Attaching package: 'ggthemes'  
##  
## The following object is masked from 'package:cowplot':  
##  
## theme_map
```

```
here()
```

```
## [1] "/home/guest/R/EDE_Fall2025"
```

```
#2 Reading in datasets
```

```
PeterPaul.chem.nutrients <- read.csv(  
  here('./Data/Processed_KEY/NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Processed.csv'),  
  stringsAsFactors = TRUE)  
NIWO_litter <- read.csv(  
  here('./Data/Processed_KEY/NEON_NIWO_Litter_mass_trap_Processed.csv'),  
  stringsAsFactors = TRUE)
```

```
#Fixing dates
```

```
PeterPaul.chem.nutrients$sampdate <- ymd(PeterPaul.chem.nutrients$sampdate)  
NIWO_litter$collectDate <- ymd(NIWO_litter$collectDate)  
class(PeterPaul.chem.nutrients$sampdate)
```

```
## [1] "Date"
```

```
class(NIWO_litter$collectDate)
```

```
## [1] "Date"
```

Define your theme

3. Build a theme and set it as your default theme. Customize the look of at least two of the following:

- Plot background
- Plot title
- Axis labels
- Axis ticks/gridlines
- Legend

```
#3
#Create a base plot to play with
my_theme <- theme_classic(base_size = 14, base_family = "serif") +
  theme(plot.title = element_text(color = '#4F4F4F'),
        axis.ticks = element_line(color = '#c77975'),
        axis.line = element_line(color = '#F0F0F0'),
        axis.title.x = element_text(color = '#4F4F4F'),
        axis.title.y = element_text(color = '#4F4F4F'),
        axis.text = element_text(color = '#c77975'),
        legend.text = element_text(color = '#4F4F4F'),
        legend.title = element_text(color = '#4F4F4F'),
        legend.background = element_rect(fill = '#F0F0F0'),
        plot.background = element_rect(fill = '#F0F0F0'),
        panel.background = element_rect(fill = '#c77975'),
        panel.grid.major = element_line(color = "#F0F0F0"), # main grid lines
        panel.grid.minor = element_line(color = "#F0F0F0"),
        rect = element_rect(color = "#767676"))

# Theme building layout used
# my_theme <- theme_base() +
# theme(
  #line =                element_line(),
  #rect =                element_rect(),
  #text =               element_text(),

  # Modified inheritance structure of text element
  #plot.title =         element_text(),
  #axis.title.x =       element_blank(),
  #axis.title.y =       element_blank(),
  #axis.text =          element_text(),

  # Modified inheritance structure of line element
  #axis.ticks =         element_line(),
  #panel.grid.major =   element_line(),
  #panel.grid.minor =   element_blank(),

  # Modified inheritance structure of rect element
  #plot.background =    element_rect(),
  #panel.background =   element_rect(),
  #legend.key =         element_rect(),

  # Modifiying legend.position
  #legend.position = 'right',
```

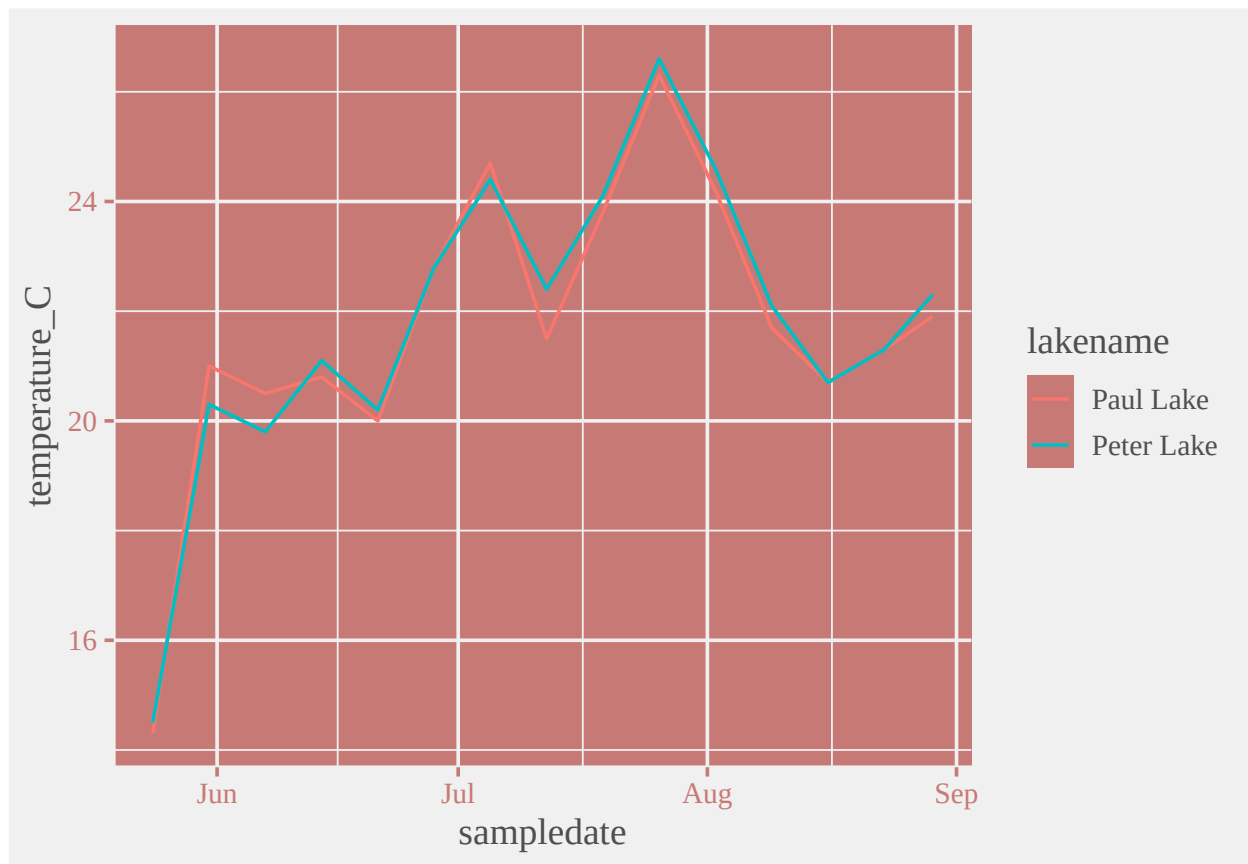
```

#complete = TRUE
# )

#Sample plot to play around with theme

PeterPaul.chem.nutrients %>%
  filter(depth == 0 & year4 == 1999) %>%
  ggplot(
    mapping = aes(
      x=sampleddate,
      y=temperature_C,
      color=lakename)
  ) +
  geom_line() +
  my_theme

```



```

# setting my_theme to be the default theme

set_theme(my_theme)

```

Create graphs

For numbers 4-7, create ggplot graphs and adjust aesthetics to follow best practices for data visualization. Ensure your theme, color palettes, axes, and additional aesthetics are edited accordingly.

4. [NTL-LTER] Plot total phosphorus (tp_ug) by phosphate (po4), with separate aesthetics for Peter and Paul lakes. Add line(s) of best fit using the `lm` method. Adjust your axes to hide extreme values (hint: change the limits using `xlim()` and/or `ylim()`).

```
#4
lake_colors <- c(
  "Peter Lake" = "#84665D",
  "Paul Lake"  = "#F6E5E3")

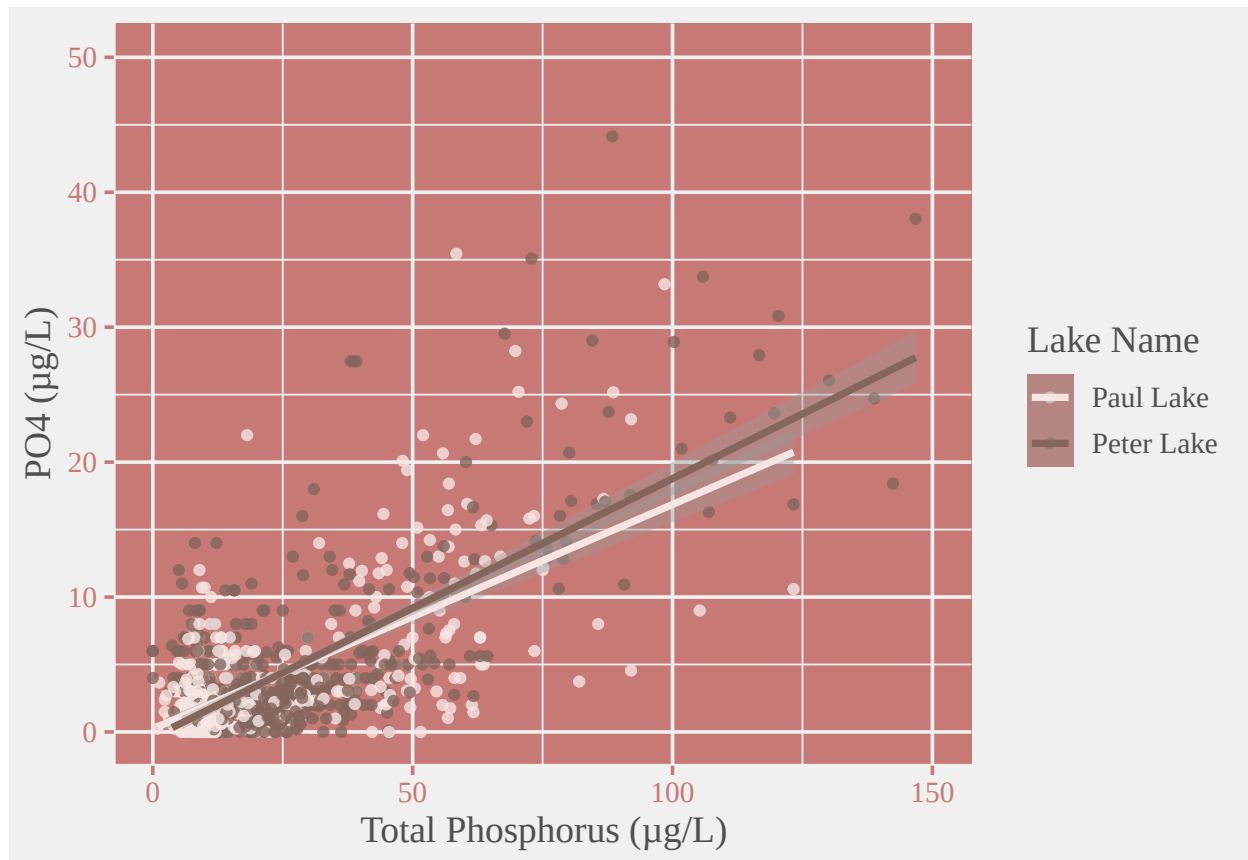
PeterPaul.chem.nutrients %>%
  ggplot(mapping = aes(
    x=tp_ug,
    y=po4,
    color=lakename)) +
  xlim(0, 150) +
  ylim(0, 50) +
  geom_point(size = 1.3, alpha = 0.8) +
  geom_smooth(method = lm) +
  scale_color_manual(values = lake_colors) +
  labs(color = "Lake Name") +
  xlab("Total Phosphorus (µg/L)") +
  ylab("P04 (µg/L)")
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```
## Warning: Removed 21948 rows containing non-finite outside the scale range
## ('stat_smooth()').
```

```
## Warning: Removed 21948 rows containing missing values or values outside the scale range
## ('geom_point()').
```

```
## Warning: Removed 2 rows containing missing values or values outside the scale range
## ('geom_smooth()').
```



5. [NTL-LTER] Make three separate boxplots of (a) temperature, (b) TP, and (c) TN, with month as the x axis and lake as a color aesthetic. Then, create a cowplot that combines the three graphs. Make sure that only one legend is present and that graph axes are aligned.

Tips: * Recall the discussion on factors in the lab section as it may be helpful here. * Setting an axis title in your theme to `element_blank()` removes the axis title (useful when multiple, aligned plots use the same axis values) * Setting a legend's position to "none" will remove the legend from a plot. * Individual plots can have different sizes when combined using `cowplot`.

```
#5
#identifying class of months
class(PeterPaul.chem.nutrients$month)
```

```
## [1] "integer"
```

```
#ensuring all 12 are here
unique(PeterPaul.chem.nutrients$month)
```

```
## [1] 5 6 7 8 9 10 11 2
```

```
#building the plot (a) with factors
p_a <- PeterPaul.chem.nutrients %>%
  ggplot(aes(x = factor(month,levels=1:12,labels=month.abb), y = temperature_C, color = lakenam)) +
  geom_boxplot() +
```

```

my_theme +
  #scale_x_discrete(name='Month', drop=FALSE) +
  scale_color_manual(values = lake_colors) +
  ylab("Temperature C°") +
  xlab("Month") +
  labs(color = "Lake Name", title = "Temperature Boxplot by Lake per Month")

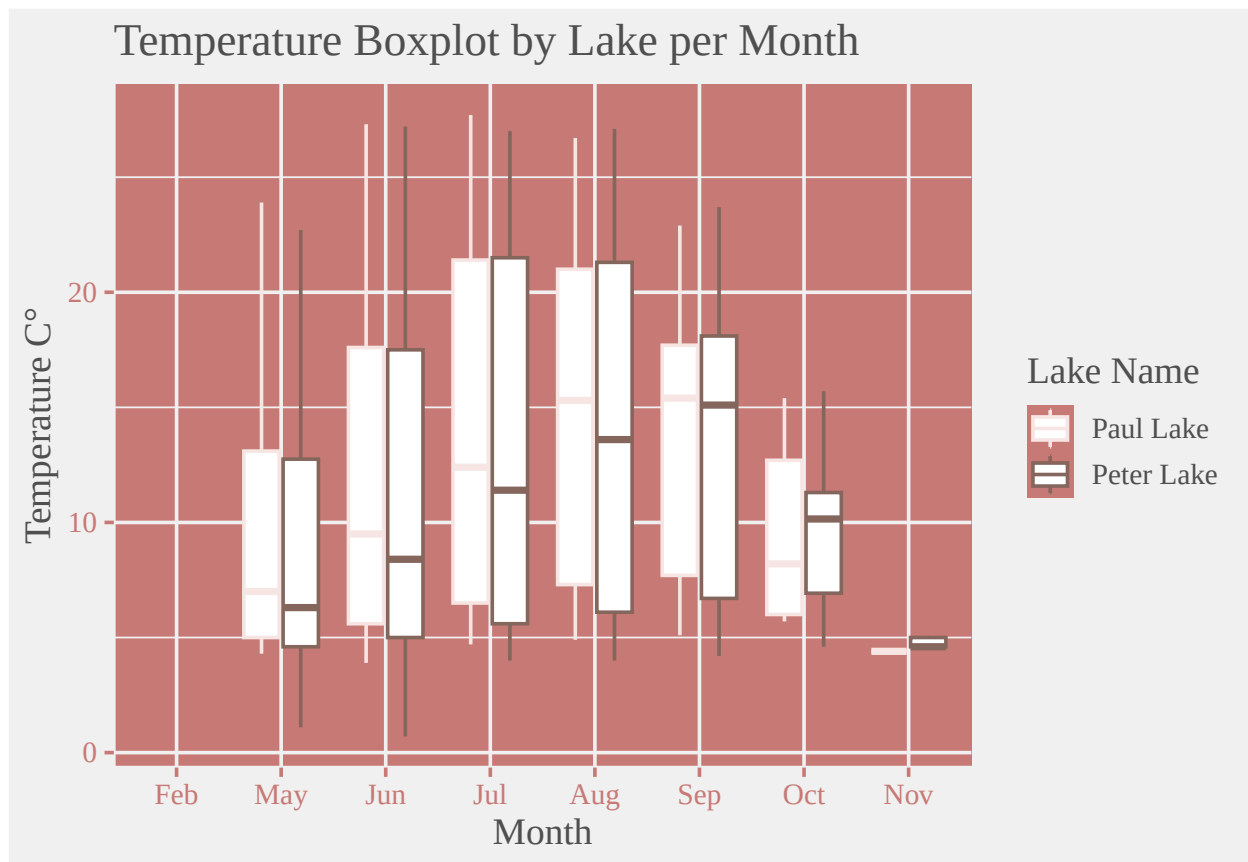
print(p_a)

```

```

## Warning: Removed 3566 rows containing non-finite outside the scale range
## ('stat_boxplot()').

```



```

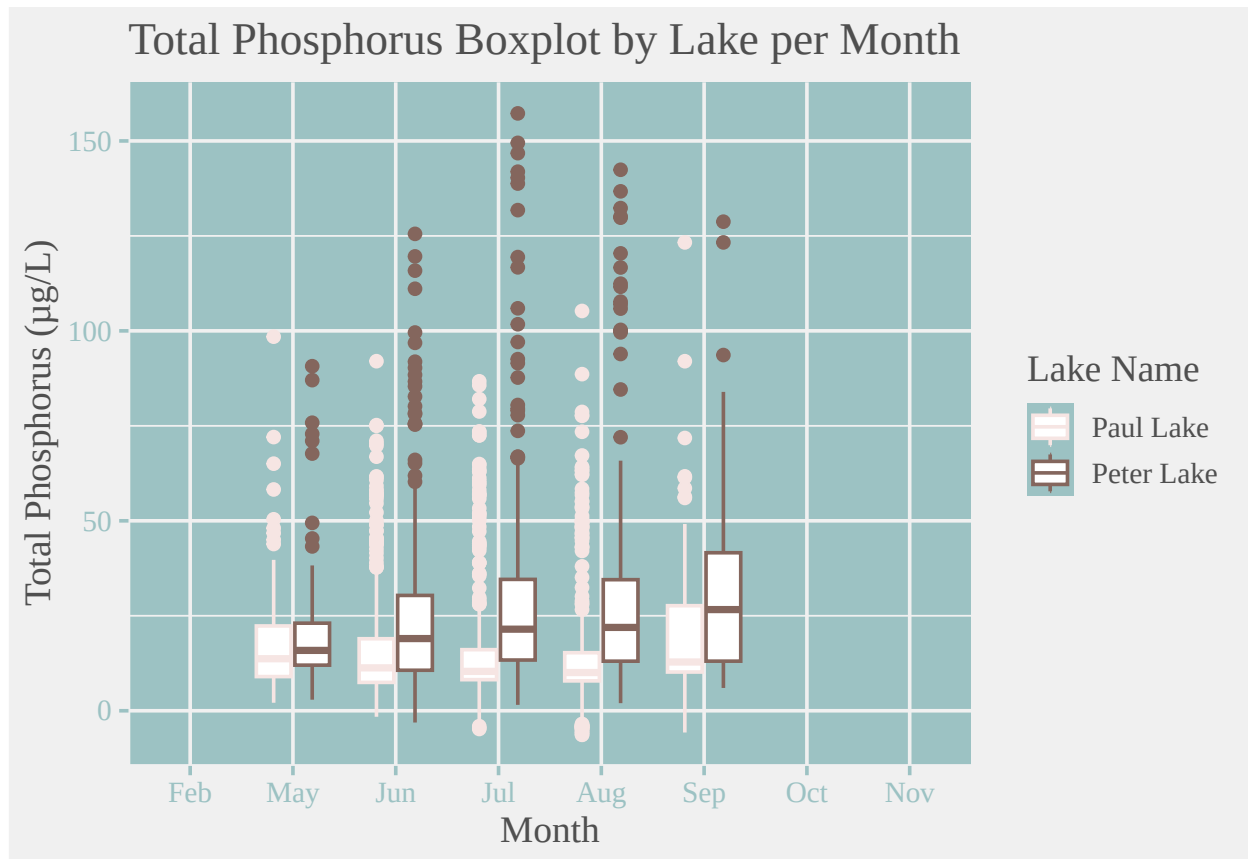
#building the plot (b) with factors
p_b <- PeterPaul.chem.nutrients %>%
  ggplot(aes(x = factor(month,levels=1:12,labels=month.abb), y = tp_ug, color = lakename)) +
  geom_boxplot() +
  my_theme +
  #scale_x_discrete(name='Month', drop=FALSE) +
  scale_color_manual(values = lake_colors) +
  ylab("Total Phosphorus (µg/L)") +
  xlab("Month") +
  labs(color = "Lake Name", title = "Total Phosphorus Boxplot by Lake per Month") +
  theme(panel.background = element_rect(fill = '#9cc2c3'),
        axis.ticks = element_line(color = '#9cc2c3'),

```

```
axis.text = element_text(color = '#9cc2c3'))

print(p_b)
```

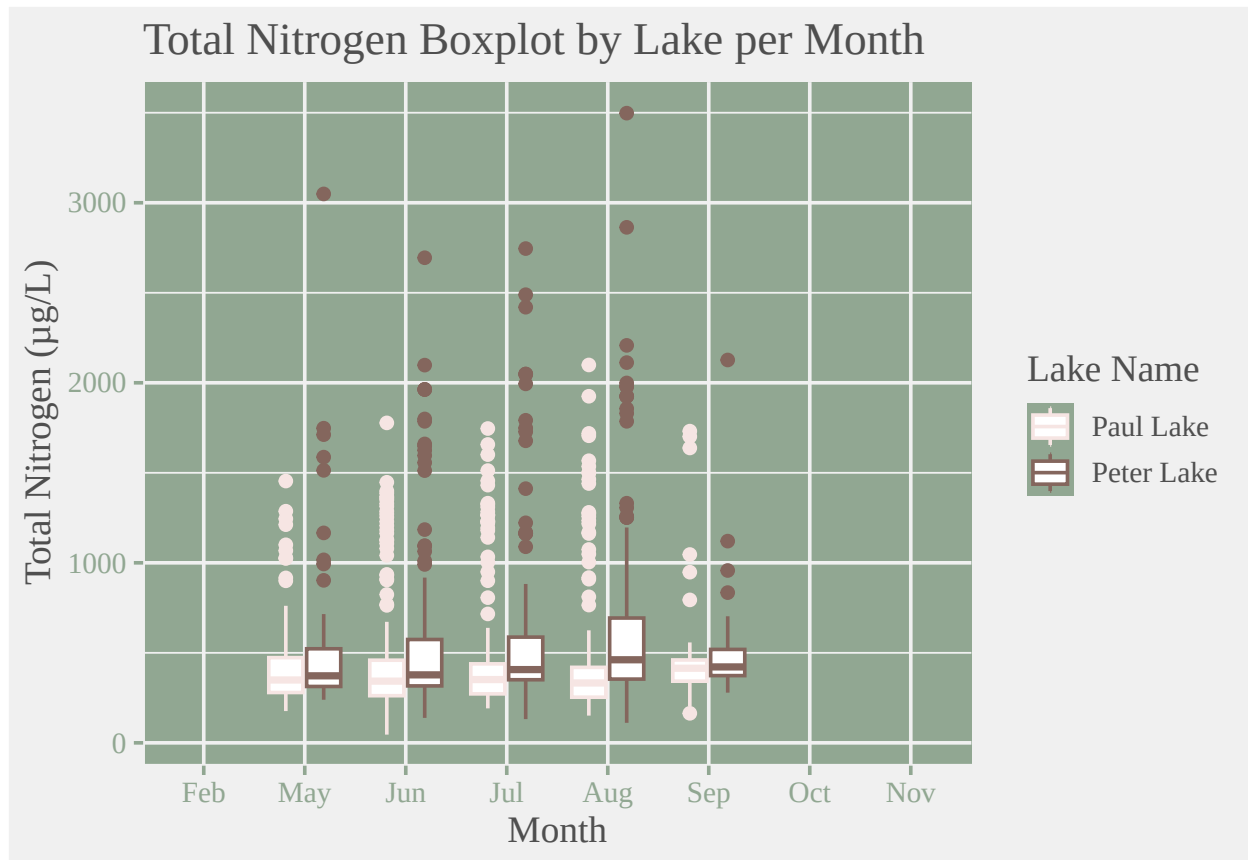
```
## Warning: Removed 20729 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```



```
#building the plot (c) with factors
p_c <- PeterPaul.chem.nutrients %>%
  ggplot(aes(x = factor(month,levels=1:12,labels=month.abb), y = tn_ug, color = lakename)) +
  geom_boxplot() +
  my_theme +
  #scale_x_discrete(name='Month', drop=FALSE) +
  scale_color_manual(values = lake_colors) +
  ylab("Total Nitrogen (µg/L)") +
  xlab("Month") +
  labs(color = "Lake Name", title = "Total Nitrogen Boxplot by Lake per Month") +
  theme(panel.background = element_rect(fill = '#91a792'),
        axis.ticks = element_line(color = '#91a792'),
        axis.text = element_text(color = '#91a792'))

print(p_c)
```

```
## Warning: Removed 21583 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```

#combining them into a single plot using cowplot

```
p_a2 <- p_a +
  theme(axis.text.x = element_blank(),
        axis.ticks.x = element_blank(),
        axis.title.x = element_blank(),
        legend.position = "none",
        axis.title.y = element_text(size = 8)) +
  labs(title = "Monthly Variance in Nutrients at Peter and Paul Lakes")

p_b2 <- p_b +
  labs(title = NULL) +
  theme(axis.text.x = element_blank(),
        axis.ticks.x = element_blank(),
        axis.title.x = element_blank(),
        axis.title.y = element_text(size = 8))

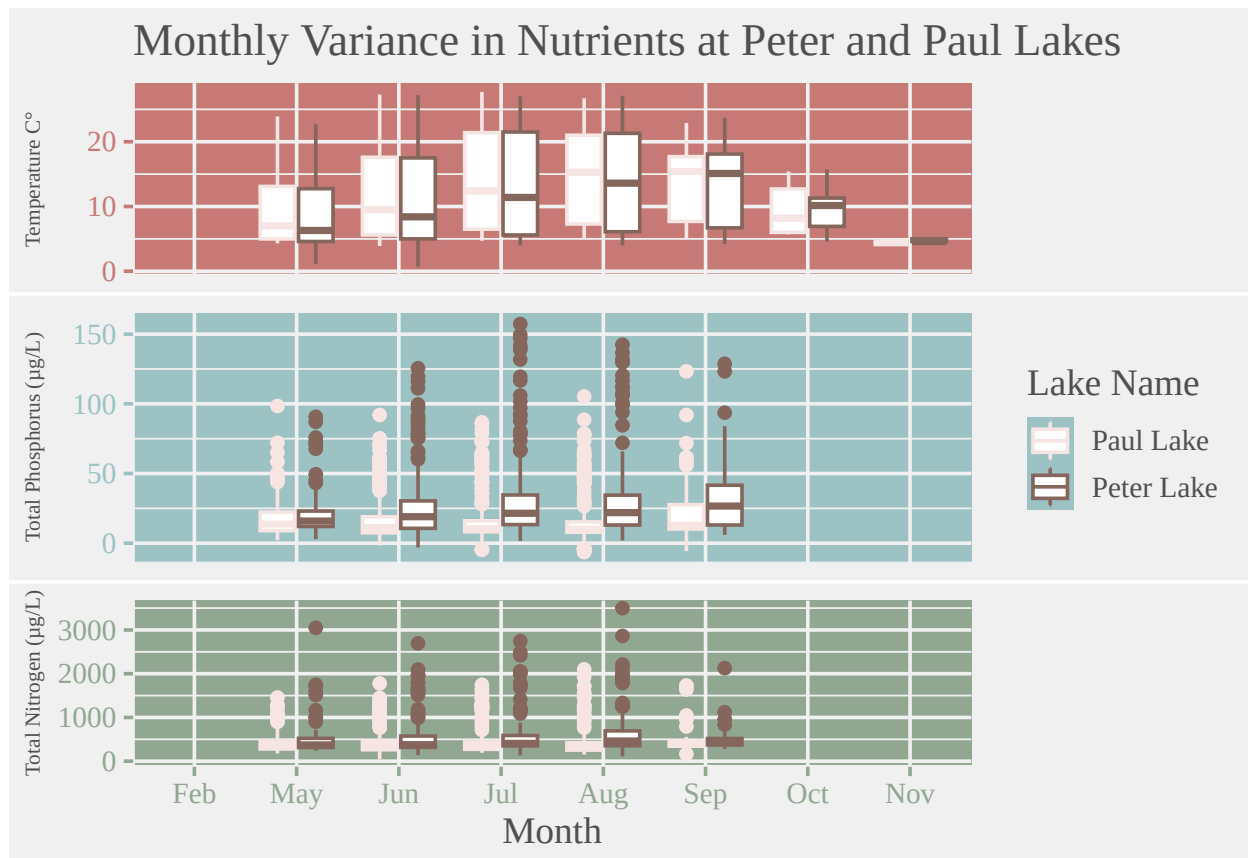
# Remove title only from p3
p_c2 <- p_c +
  labs(title = NULL) +
  theme(legend.position = "none",
        axis.title.y = element_text(size=8, margin = margin(t = -5, r = 5, b = 10, l = 0)))

plot_grid(p_a2, p_b2, p_c2,
          ncol = 1,
          align = "v")
```

```
## Warning: Removed 3566 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```

```
## Warning: Removed 20729 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```

```
## Warning: Removed 21583 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```



Question: What do you observe about the variables of interest over seasons and between lakes?

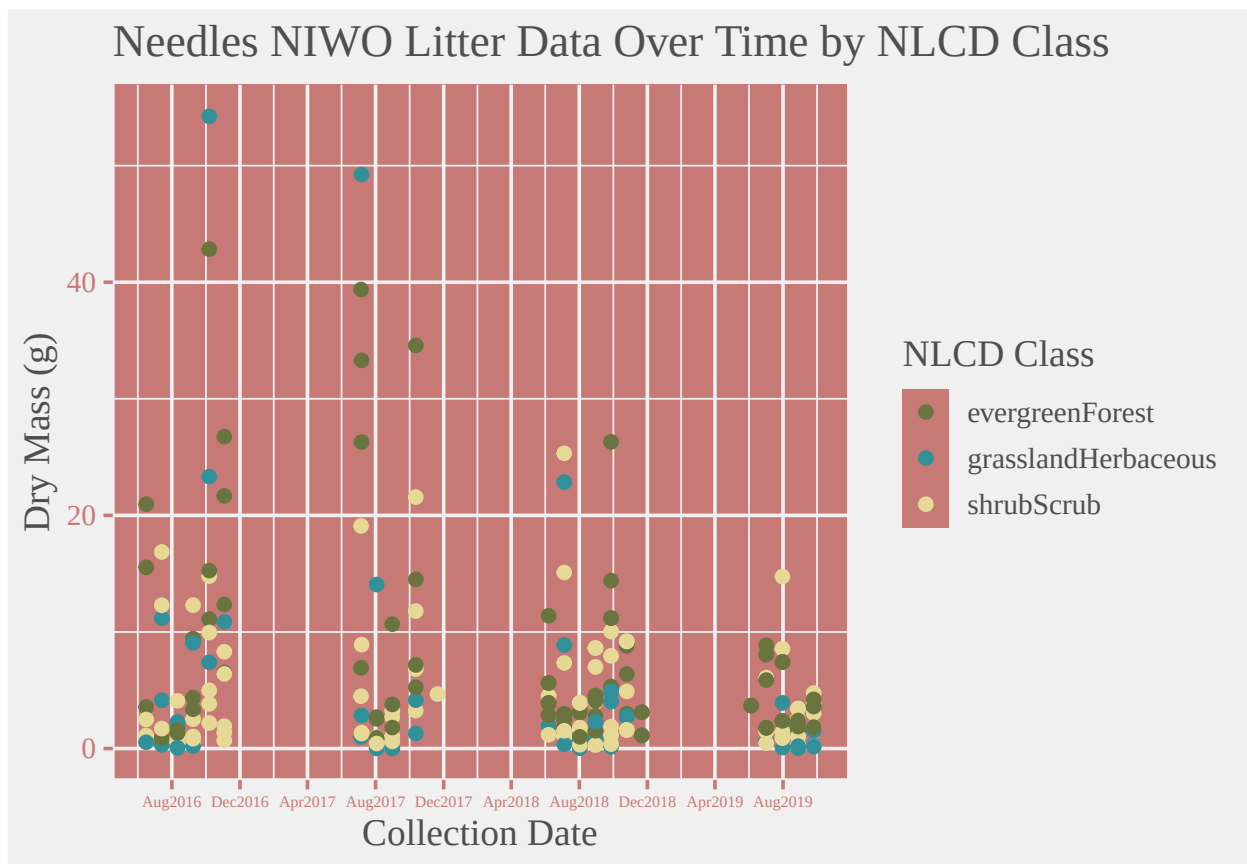
Answer: Generally, concentrations of TP and TN appear to rise and spike minutely during the summer months, peaking around July and August. They start to fall again during September. Temperature, of course rises during the summer months. Although TP and TN could be related to temperature, they could also be related to water stratification or unrelated events that occur during the summer. More analysis is needed to understand this phenomenon.

6. [Niwot Ridge] Plot a subset of the litter dataset by displaying only the “Needles” functional group. Plot the dry mass of needle litter by date and separate by NLCD class with a color aesthetic. (no need to adjust the name of each land use)
7. [Niwot Ridge] Now, plot the same plot but with NLCD classes separated into three facets rather than separated by color.

#6 Litter Subplot separated by color

```
litter_colors <- c(
  "evergreenForest" = "#6a743f",
  "grasslandHerbaceous" = "#32909a",
  "shrubScrub" = "#e6d995")

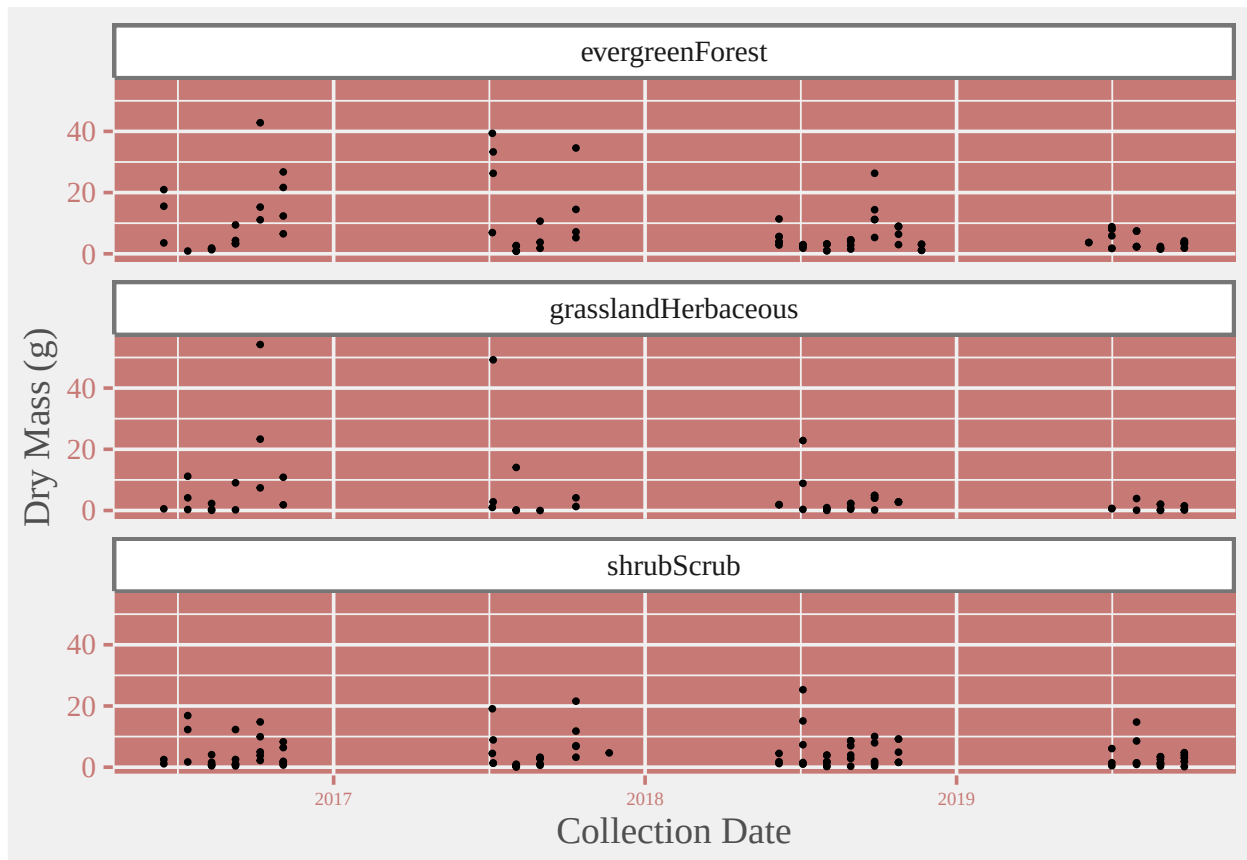
NIWO_litter %>%
  filter(functionalGroup == 'Needles') %>%
  ggplot(mapping = aes(x=collectDate, y=dryMass, color=nlcdClass)) +
  geom_point() +
  xlab("Collection Date") +
  ylab("Dry Mass (g)") +
  labs(color = 'NLCD Class', title = "Needles NIWO Litter Data Over Time by NLCD Class") +
  scale_x_date(date_breaks = "4 months", date_labels = "%b%Y") +
  scale_color_manual(values = litter_colors) +
  theme(axis.text.x = (element_text(size=6)))
```



#7 Facets

```
NIWO_litter %>%
  filter(functionalGroup == 'Needles') %>%
  ggplot(mapping = aes(x=collectDate, y=dryMass)) +
  geom_point(size = 0.5) +
  xlab("Collection Date") +
  ylab("Dry Mass (g)") +
```

```
# scale_x_date(date_breaks = "4 months", date_labels = "%b%Y") + #removing scale breaks with faceting
theme(axis.text.x = (element_text(size=7))) +
facet_wrap(vars(nlcdClass), nrow = 3)
```



Question: Which of these plots (6 vs. 7) do you think is more effective, and why?

Answer: Plot 7 is more effective from a data visualization perspective, because you are visualizing comparisons more easily. With different colors on the same plot, the shape of the data is less easily visible because there are other colored points on the same plot. When faceting for so many points, it's easier to see if there is a different curve or shape to the data. However, some may prefer being able to see data compared on the same plot. If the data were visualized as lines, it may be more helpful, but for point data, I believe seeing the shape of the points on different facets is more effective.